

TD1 : POO JAVA (J2SE, J2ME ET J2EE)

Par : Kevin Fonkoua.

Partie 1 – Classe de base et encapsulation

1. Créez une classe `Produit` avec les attributs privés suivants :
 - `nom` (String)
 - `prixHT` (double)
 - `quantiteStock` (int)
2. Ajoutez un constructeur permettant d'initialiser ces trois attributs.
Le constructeur doit vérifier que le prix HT et la quantité sont positifs (sinon, on les met à 0).
3. Pour respecter l'encapsulation, fournissez les getters et setters pour chaque attribut.
Dans le setter de `prixHT` et de `quantiteStock`, vérifiez à nouveau la validité des valeurs.
4. Ajoutez une méthode double `calculerPrixTTC(double tva)` qui retourne le prix TTC ($\text{prixHT} * (1 + \text{tva})$).
La TVA est passée en paramètre (exemple : 0.20 pour 20%).
5. Ajoutez une méthode void `afficherInfos()` qui affiche toutes les informations du produit (nom, prix HT, quantité, et prix TTC avec une TVA par défaut de 20%).

Partie 2 – Héritage et polymorphisme

6. Créez deux sous-classes :
 - `ProduitAlimentaire` : ajoute un attribut `datePeremption` (String).
Redéfinissez la méthode `afficherInfos()` pour qu'elle affiche aussi la date de péremption.
 - `ProduitElectronique` : ajoute un attribut `dureeGarantie` (en mois, entier).
Redéfinissez `afficherInfos()` pour afficher la durée de garantie.
7. Dans chaque sous-classe, redéfinissez éventuellement `calculerPrixTTC` si le taux de TVA est différent (par exemple, 5.5% pour l'alimentaire, 20% pour l'électronique).
Vous pouvez choisir de passer la TVA en paramètre ou de la définir en constante dans la classe.
8. Créez une méthode `main` dans une classe `Test` pour tester le polymorphisme :
 - Déclarez un tableau de `Produit` contenant au moins un objet de chaque type.
 - Parcourez ce tableau et appelez `afficherInfos()` sur chaque élément.
Constatez que la bonne version de la méthode est appelée.

Partie 3 – Classe abstraite

9. Transformez la classe `Produit` en **classe abstraite**.
 - Rendez la méthode `afficherInfos()` abstraite (chaque sous-classe devra donc la définir).

- Ajoutez une méthode concrète `boolean estDisponible()` qui retourne `true` si la quantité en stock est supérieure à 0.
- 10. Adaptez les sous-classes pour qu'elles implémentent `afficherInfos()` (vous l'avez déjà fait, mais maintenant c'est imposé par l'abstraction).
- 11. Vérifiez que le polymorphisme fonctionne toujours.

Partie 4 – Interface

12. Définissez une interface `Stockable` contenant les méthodes :
 - `void ajouterStock(int quantite)`
 - `void retirerStock(int quantite)` (qui peut lever une exception si stock insuffisant – nous le ferons plus tard)
 - `int getStock()`
13. Faites en sorte que la classe `Produit` (abstraite) implémente cette interface.
*Les méthodes `ajouterStock` et `retirerStock` seront concrètes dans `Produit` (elles modifieront `quantiteStock`).
 La méthode `getStock()` est déjà un getter existant.*
14. Dans `retirerStock`, si la quantité à retirer est supérieure au stock disponible, affichez simplement un message d'erreur pour l'instant (nous utiliserons une exception ensuite).
15. Testez ces nouvelles méthodes dans le `main`.

Partie 5 – Gestion des exceptions

16. Créez une **exception personnalisée** `StockInsuffisantException` qui hérite de `Exception`.
Elle pourra avoir un constructeur prenant un message.
17. Modifiez la méthode `retirerStock` de l'interface `Stockable` pour qu'elle déclare `throws StockInsuffisantException`.
Dans l'implémentation, si la quantité demandée dépasse le stock, levez cette exception avec un message approprié.
18. Créez une autre exception `PrixInvalideException` (héritant de `RuntimeException` ou `Exception`).
Levez-la dans le constructeur de `Produit` et dans le setter de `prixHT` si le prix est négatif.
19. Adaptez votre code pour traiter ces exceptions :
 - Dans le `main`, lors de la création d'un produit avec un prix négatif, attrapez l'exception et affichez un message.
 - Lors de l'appel à `retirerStock`, utilisez un bloc `try-catch` pour gérer le cas de stock insuffisant.
20. Enfin, ajoutez une méthode `void transfererStock(Produit destination, int quantite)` dans votre classe `Test` (ou dans une classe utilitaire) qui tente de retirer la quantité du produit courant et de l'ajouter au produit destination.
Gérez les exceptions potentielles.

Partie 6 – Attributs de classe (statiques)

1. Dans la classe `Produit`, ajoutez un **attribut de classe** `compteur` qui s'incrémente à chaque création d'un nouveau produit.

- Cet attribut sera `private static int compteur`.
- Ajoutez un getter statique `public static int getCompteur()`.
- 2. Ajoutez également un attribut statique `tvaParDefaut` (de type `double`) qui représente le taux de TVA par défaut pour tous les produits (par exemple 20%).
 - Pourrez le modifier via un setter statique.
- 3. Modifiez la méthode `calculerPrixTTC()` pour qu'elle utilise `tvaParDefaut` si aucun taux n'est passé en paramètre (surcharge de méthode).
- 4. Dans le constructeur de `Produit`, utilisez le compteur pour afficher un message à chaque création :


```
"Création du produit n°" + compteur.
```

Partie 7 – Design pattern : Fabrique (Factory)

Pour centraliser la création d'objets `Produit`, nous allons implémenter le pattern **Fabrique**.

5. Créez une classe **`ProduitFactory`** avec une méthode statique :

```
public static Produit creerProduit(String type, String nom, double
prix, int stock, String... options)
```

- Le paramètre `type` peut être "alimentaire" ou "electronique".
 - Les `options` contiennent la date de péremption pour l'alimentaire, ou la durée de garantie pour l'électronique.
 - La méthode doit lever une `IllegalArgumentException` si le type est inconnu ou si les options sont manquantes.
6. La fabrique doit également incrémenter le compteur et affecter le taux de TVA par défaut (si vous ne l'avez pas déjà fait dans le constructeur).
 7. Testez la fabrique dans le `main` en créant plusieurs produits sans appeler explicitement `new`.

Partie 8 – Design pattern : Observateur (Observer)

Pour illustrer le pattern **Observer**, nous allons simuler une alerte de stock.

8. Définissez une interface `ObservateurStock` avec une méthode `void alerteStockFaible(Produit p, int stockRestant)`.
9. Dans la classe `Produit`, ajoutez une liste d'observateurs (`List<ObservateurStock>`) et des méthodes pour ajouter/supprimer un observateur.
10. Dans la méthode `retirerStock`, après la diminution du stock, si le stock passe en dessous d'un seuil (par exemple 5), notifiez tous les observateurs.
11. Créez une classe `AlerteLogger` qui implémente `ObservateurStock` et se contente d'afficher un message dans la console.
12. Dans le `main`, attachez un `AlerteLogger` à quelques produits et testez le retrait de stock.

Partie 9 – Main complet (démonstration de toutes les notions)

Écrivez une classe `MainFinale` avec une méthode `main` qui :

- Affiche le nombre initial de produits (via le compteur statique).
- Modifie la TVA par défaut (via le setter statique).
- Utilise la `ProduitFactory` pour créer :
 - un produit alimentaire (pain, 1.0€, stock 10, date "31/12/2025")
 - un produit électronique (téléphone, 500€, stock 3, garantie 24 mois)
 - un produit alimentaire avec un prix négatif (doit déclencher une exception)
- Attrape les exceptions liées aux prix invalides.
- Affiche les informations de chaque produit en utilisant le polymorphisme (boucle sur un tableau de `Produit`).
- Ajoute un observateur de type `AlerteLogger` au produit électronique.
- Effectue des retraits de stock sur le produit électronique :
 - retire 2 unités (stock passe à 1, pas d'alerte)
 - retire 1 unité (stock passe à 0, alerte déclenchée si seuil à 5 ? Ici seuil=5 donc alerte)
 - essaie de retirer 10 unités (doit lever `StockInsuffisantException`)
- Pour le produit alimentaire, transférez 5 unités vers un autre produit (en utilisant la méthode `transfererStock` de l'étape 20 de la partie 5).
- Enfin, affichez le nombre total de produits créés (via le compteur statique).